

EXPERIMENT – 1.2

1. **AIM:** To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

2. **OBJECTIVE:**

1. To manage draft data using frontend state
2. To implement CRUD operations for draft handling
3. To understand asynchronous workflows in UI applications
4. To optionally simulate backend communication using mock APIs

3. **SOFTWARE REQUIREMENTS:**

1. Node.js & npm
2. React.js
3. Code Editor
4. Web Browser
5. Mock API tools

4. **THEORY**

A **Draft Management System** is a frontend feature that allows users to temporarily save their work before publishing it. It provides flexibility by enabling users to create, edit, retrieve, and delete drafts at any time. This functionality is commonly used in applications such as email clients, blogging platforms, content management systems, and social media applications, where users may not want to publish their content immediately.

In modern web development, frontend modules are often developed independently before integrating with backend services. React.js provides an efficient way to build such applications using reusable components and state management. In this experiment, the draft management system is implemented entirely on the frontend using React Hooks and browser storage mechanisms. This approach helps developers focus on user interface design and application logic without depending on backend infrastructure.

The application uses **React's useState() Hook** to maintain the list of drafts and the current draft content. Whenever the user enters text and clicks the **Save Draft** button, a new draft is created and stored in the application's state. The state automatically updates the user interface without requiring a page reload. This demonstrates React's reactive programming model, where changes in state automatically trigger component re-rendering.

To ensure that drafts are not lost after refreshing the browser, **Local Storage** is used as a client-side persistence mechanism. Local Storage stores data in the browser as key-value pairs and retains the information even after the browser is closed or refreshed. The application retrieves previously saved drafts when it starts and updates Local Storage whenever drafts are added, edited, or deleted. This provides users with a seamless experience and prevents accidental data loss.

5. IMPLEMENTATION/PROCEDURE

1. Create a React application using Create React App.
2. Design a draft data structure using React state.
3. Create an input field to enter draft content.
4. Implement **Save Draft (Create)** functionality.
5. Display all saved drafts dynamically.
6. Implement **Edit Draft (Update)** functionality.
7. Implement **Delete Draft (Delete)** functionality.
8. Store drafts in Local Storage using `localStorage.setItem()`.
9. Load saved drafts using `useEffect()` and `localStorage.getItem()`.
10. Update the UI automatically whenever the draft state changes.
11. Run the application using `npm start`.

6. CODE

```
import { useState } from "react";
```

```
function App() {  
  const [platform, setPlatform] = useState("Twitter");  
  const [post, setPost] = useState("");  
  const [drafts, setDrafts] = useState([]);
```

```
  const limits = {  
    Twitter: 280,  
    LinkedIn: 3000,  
    Instagram: 2200,  
  };
```

```
  const limit = limits[platform];  
  const isExceeded = post.length > limit;
```

```
  const saveDraft = () => {  
    if (post.trim() === "") {  
      alert("Please write something first!");  
      return;  
    }  
  }
```

```
  setDrafts([...drafts, post]);  
  setPost("");  
};
```

```
const editDraft = (index) => {  
  setPost(drafts[index]);
```

```

const updatedDrafts = drafts.filter((_, i) => i !== index);
setDrafts(updatedDrafts);
};

const deleteDraft = (index) => {
  const updatedDrafts = drafts.filter((_, i) => i !== index);
  setDrafts(updatedDrafts);
};

return (
  <div
    style={{
      backgroundColor: "#f4f6f9",
      minHeight: "100vh",
      padding: "40px",
      fontFamily: "Arial",
    }}
  >
    <div
      style={{
        backgroundColor: "white",
        width: "650px",
        margin: "auto",
        padding: "30px",
        borderRadius: "15px",
        boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
      }}
    >
      <h1
        style={{
          textAlign: "center",
          color: "#2c3e50",
          marginBottom: "30px",
        }}
      >
         Social Media Post Composer
      </h1>

      <label>
        <b>Select Platform</b>
      </label>

      <br />
      <br />

```

```
<select
  value={platform}
  onChange={(e) => setPlatform(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
  }}
>
  <option>Twitter</option>
  <option>LinkedIn</option>
  <option>Instagram</option>
</select>
```

```
<br />
```

```
<br />
```

```
<label>
  <b>Write Your Post</b>
</label>
```

```
<br />
```

```
<br />
```

```
<textarea
  rows="6"
  placeholder="Type your post here..."
  value={post}
  onChange={(e) => setPost(e.target.value)}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
  }}
></textarea>
```

```
<br />
```

```
<br />
```

```
<p
```

```

    style={{
      fontWeight: "bold",
      color: isExceeded ? "red" : "#2c3e50",
    }}
  >
  Characters: {post.length} / {limit}
</p>

```

```

{isExceeded ? (
  <p
    style={{
      color: "red",
      fontWeight: "bold",
    }}
  >
    + Character limit exceeded!
  </p>
) : (
  <p
    style={{
      color: "green",
      fontWeight: "bold",
    }}
  >
     Ready to post
  </p>
)}

```

```

<button
  onClick={saveDraft}
  style={{
    backgroundColor: "#007bff",
    color: "white",
    border: "none",
    padding: "12px 20px",
    borderRadius: "8px",
    cursor: "pointer",
    fontSize: "16px",
    marginTop: "10px",
  }}
>
   Save Draft
</button>

```

```
<hr style={{ margin: "30px 0" }} />
```

```
<h2>📌 Saved Drafts</h2>
```

```
{drafts.length === 0 ? (
```

```
  <p>No drafts available.</p>
```

```
) : (
```

```
  drafts.map((draft, index) => (
```

```
    <div
```

```
      key={index}
```

```
      style={{
```

```
        backgroundColor: "#f8f9fa",
```

```
        padding: "15px",
```

```
        borderRadius: "10px",
```

```
        marginBottom: "15px",
```

```
        boxShadow: "0px 2px 8px rgba(0,0,0,0.1)",
```

```
    }}>
```

```
    <
```

```
      <p>{draft}</p>
```

```
    <button
```

```
      onClick={() => editDraft(index)}
```

```
      style={{
```

```
        backgroundColor: "#28a745",
```

```
        color: "white",
```

```
        border: "none",
```

```
        padding: "8px 15px",
```

```
        borderRadius: "5px",
```

```
        cursor: "pointer",
```

```
        marginRight: "10px",
```

```
      }}>
```

```
    <
```

```
      📌 Edit
```

```
    </button>
```

```
  <button
```

```
    onClick={() => deleteDraft(index)}
```

```
    style={{
```

```
      backgroundColor: "#dc3545",
```

```
      color: "white",
```

```
      border: "none",
```

```
      padding: "8px 15px",
```

```

        borderRadius: "5px",
        cursor: "pointer",
      }}
    >
    🗑 Delete
  </button>
</div>
))
}}
</div>
</div>
);
}
export default App;

```

7. OUTPUT

The Draft Management System was successfully implemented using React.js. Users can create, view, edit, and delete drafts through an interactive user interface. Draft information is managed using React state and is automatically updated whenever changes are made. The application stores drafts in the browser's Local Storage, allowing them to persist even after the page is refreshed. The system demonstrates proper implementation of CRUD operations, dynamic rendering, and frontend state management, providing a responsive and user-friendly draft management experience.

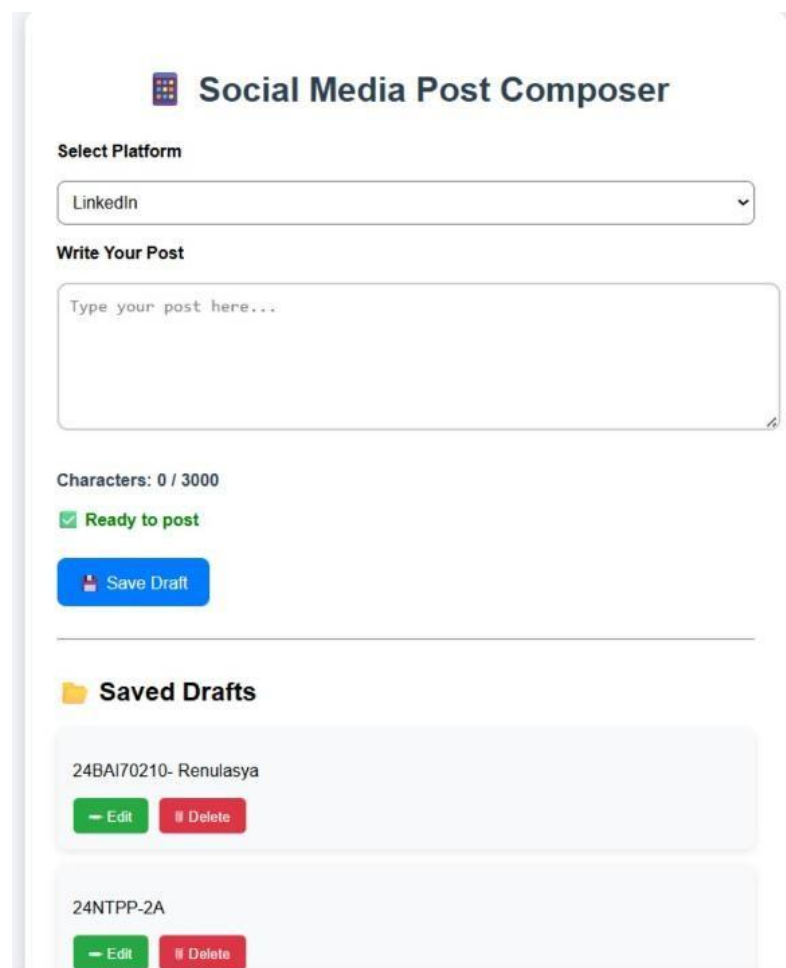


Fig 1.1: Post Composer UI with Platform Constraints with draft saving management

8. LEARNING OUTCOME

- Understand frontend state management using React.
- Implement CRUD operations for draft management.
- Use useState() and useEffect() Hooks effectively.
- Store and retrieve data using Local Storage.
- Build responsive and dynamic user interfaces.
- Understand how frontend applications manage temporary data before backend integration.
- Gain practical experience in developing real-world React applications.